

Things I would not skip

A short list. All learned the hard way. Sometimes painfully.

- Pin your model versions. Use specific tags like *llama3.1:8b-instruct-q4_K_M*. Not 'latest'. The day a model behaves oddly after a silent update, you will thank me.
- Use named volumes for model storage. Models are huge files. Treat that cache like gold.
- Watch GPU memory and response latency. Set alerts before your users notice anything wrong.
- Put rate limits at the gateway. Even your own staff can accidentally hammer an endpoint and bring it down.
- Use quantized models when you can. Q4 or Q5 versions often give you 95 percent of the quality at a fraction of the memory.
- Run separate Ollama instances for inference and embedding if your traffic is heavy. Long generations should not block embedding calls.
- Schedule a regular review for new model releases. New ones drop constantly. Pick a cadence. Do not chase every release.
- Put authentication in front of Ollama. By default, there is none. I have seen teams forget this. It is not a fun day when it goes wrong.
- Plan for scale on day one. Build your app as if it talks to a pool of Ollama instances. Even if you start with one.

Where I think this is going

A few honest predictions for a two to three-year horizon based on what I see in the field.

Open models will keep narrowing the gap with the big paid ones. The pace of progress over the last eighteen months has genuinely surprised me. Does not look like slowing.

Hardware will get more friendly. Consumer GPUs with more VRAM. New chip designs like Apple Silicon and AMD's unified memory are becoming real options for medium models.

Tooling will mature. Better scaling, easier fine-tuning, standardised monitoring, tighter integration with vector databases, and agent frameworks. Ollama itself is moving fast. Projects like vLLM, TensorRT-LLM, and llama.cpp keep pushing efficiency.

The biggest shift, in my honest view, is that enterprise AI is going hybrid. Paid APIs for cutting-edge or low-volume work where peak quality matters. Self-hosted models for high-volume, sensitive, cost-conscious work. Mixed setups will be the default. Anyone selling you a one-or-the-other story is selling you something else.

A few closing thoughts

Looking back at the last two years, I have to admit I find this exciting.

We went from "you need a PhD and a cloud contract to run a real AI model" to "spin one up in fifteen minutes with one Docker command." That is not a small change. That is the kind of shift that opens up new possibilities for teams that were locked out earlier.

Ollama and Docker together are a great example of what I have always loved about open source. Practical tools built well. Quietly remove the friction. Put real power in the hands of developers and the businesses they work for. Good infrastructure should be invisible. These tools are getting close.

If you have been waiting for the right moment to bring AI in-house, I think the wait is over. Pick a use case. Find a server. Write a *Compose* file. Start building. The cost of entry has never been lower. The strategic value of owning your AI stack has never been higher.

The future of enterprise AI will not be built in someone else's data centre. It will be built on infrastructure your team controls, with tools your team understands, using models your team can look inside.

Ollama and Docker are quietly making that future real, one container at a time. **END** 🐧

By: Vishal Tanna

The author is the CEO of TechCrave Solutions, a technology services company that helps enterprises adopt open source, cloud native, and AI-driven solutions. He has over 15 years of experience in software development and technology leadership, with a particular interest in scalable architectures and digital transformation.

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

The latest from the Open Source world is here.

OpenSourceForU.com

Join the community at facebook.com/opensourceforu

Follow us on Twitter @OpenSourceForU